

Was ist Scrum?

Scrum ist eines der beliebtesten agilen Frameworks, das Teams dabei hilft, komplexe Projekte zu bewältigen, indem es die Arbeit in kleinere, iterative Zyklen, sogenannte Sprints, unterteilt. Es wurde entwickelt, um die Zusammenarbeit zu fördern, die Transparenz zu erhöhen und kontinuierliche Verbesserungen voranzutreiben.

Ob du Software entwickelst, IT-Anfragen verwaltest oder funktionsübergreifende Projekte koordinierst, die Scrum-Methode verbindet Meetings, Tools und Rollen, die im Zusammenspiel dir und deinem Team dabei helfen, die Arbeit zu strukturieren und zu verwalten.

In diesem Leitfaden gehen wir auf die Grundlagen von Scrum ein, untersuchen seine Rollen und Praktiken und geben Tipps, wie du mit dem [agilen Projektmanagement](#) durchstarten kannst, um deine Arbeit effektiver zu planen, nachzuverfolgen und zu erledigen.

Was ist Scrum?

Scrum ist ein Framework für agiles Projektmanagement, das Teams dabei unterstützt, ihre Aufgaben mithilfe von Werten, Prinzipien und Praktiken zu strukturieren und zu verwalten. Scrum sorgt dafür, dass Teams durch Erfahrung lernen, sich selbst organisieren, während sie an einem Problem arbeiten, und über ihre Erfolge und Misserfolge nachdenken, um sich kontinuierlich zu verbessern.

Wer nutzt hauptsächlich das Scrum-Framework?

Softwareentwicklungs- und Engineering-Teams verwenden häufig die Scrum-Methode, um sich an sich ändernde Anforderungen anzupassen und die Kosten zu verwalten. Doch Entwickler und Ingenieure sind nicht die einzigen, die Scrum nutzen. Die Prinzipien und Lektionen der Methode können auf jede Art von Teamarbeit angewendet werden, von Marketing- bis hin zu IT-Teams.

Der Unterschied zwischen Agile und Scrum

Viele denken, Scrum und Agile seien dasselbe, da sich bei Scrum alles um [kontinuierliche Verbesserungen](#) dreht – eines der Kernprinzipien von Agile. Jedoch handelt es sich bei Scrum um ein Framework zur Bewältigung von Aufgaben, während Agile eine Philosophie beschreibt.

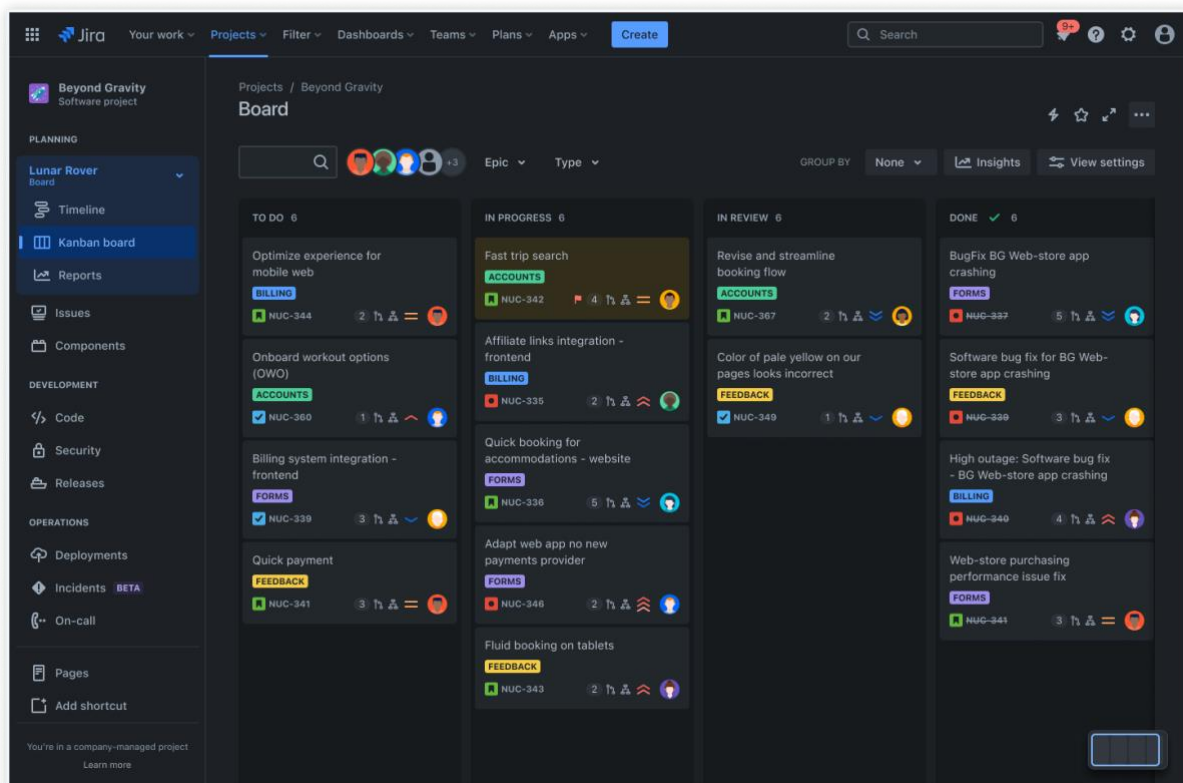
Bei der Philosophie von Agile ist der Fokus auf eine kontinuierliche und schrittweise Verbesserung durch kleine und regelmäßige Releases gelegt. Man kann als Einzelner nicht wirklich "agil werden". Die Hingabe des gesamten Teams ist nötig, um einen neuen Denkansatz zur Wertschöpfung für den Kunden zu finden.

Doch ein Framework wie Scrum kann helfen, diese neue Denkweise zu verfolgen und die tägliche Kommunikation sowie die täglichen Aufgaben auf Agile-Grundsätze auszurichten.

Worin der Unterschied zwischen Agile und Scrum liegt, kannst du im Scrum-Leitfaden und im [Agile Manifest](#) nachlesen. Im Agile Manifest werden vier Werte erläutert:

- Individuen und Interaktionen haben Vorrang vor Prozessen und Tools.
- Funktionierende Software hat Vorrang vor umfassender Dokumentation
- Die Zusammenarbeit mit den Kunden hat Vorrang vor Vertragsverhandlungen
- Das Reagieren auf Veränderungen hat Vorrang vor dem Befolgen eines Plans.

Scrum basiert auf Empirismus und der Lean-Denkweise. Empirismus steht dafür, dass Wissen mit Erfahrung einhergeht und Entscheidungen auf Grundlage von Beobachtungen getroffen werden.



Ein Diagramm des Scrum-Frameworks

Durch die [Lean-Denkweise](#), die oft zusammen mit Kanban Boards verwendet wird, konzentriert man sich verstärkt auf das Wesentliche, während Unwichtiges wegfällt. Die Scrum-Theorie bezieht sich auf diese grundlegenden Prinzipien: Empirismus, Lean-Denkweise und iterative Verbesserung.

Bei korrekter Umsetzung dient sie als Leitfaden für die Struktur, die Praktiken und die kontinuierliche Verbesserung von Scrum-Implementierungen.

Die Geschichte von Scrum mit Jeff Sutherland

Die Geschichte des Scrum-Frameworks verdankt einen Großteil ihres Erfolgs der Pionierarbeit von Jeff Sutherland. Zusammen mit Ken Schwaber entwickelte Sutherland Anfang der 1990er Jahre Scrum, um den Herausforderungen des Managements komplexer Softwareentwicklungsprojekte zu begegnen.

Sutherland brachte eine einzigartige Perspektive in das Projektmanagement ein, die Teamarbeit, Anpassungsfähigkeit und klare Kommunikation in den Vordergrund stellte. Seine Erfahrungen in Hochdruckumgebungen prägten den kollaborativen und iterativen Charakter von Scrum und machten es zu einem leistungstarken Tool für die Bewältigung komplexer Probleme.

Heute zeigt sich der Einfluss von Sutherland darin, wie Scrum-Teams ihre Arbeit organisieren, Mehrwert schaffen und sich kontinuierlich verbessern.

Das Scrum-Framework

Scrum erfordert bestimmte Rollen und Komponenten, darunter einen [Scrum Master](#), einen Produktinhaber und das Scrum-Team, um durch iterative Sprints einen Mehrwert zu erzielen.

Das Scrum-Framework umreißt eine Reihe von Werten, Prinzipien und Praktiken, die Scrum-Teams befolgen, um ein Produkt oder einen Service zu liefern. Es beschreibt die Mitglieder eines Scrum-Teams und ihre Verantwortlichkeiten oder [Scrum-Artefakte](#).

Diese Artefakte definieren das Produkt und die für seine Erstellung nötige Arbeit, während [Scrum-Zeremonien](#) das Scrum-Team durch die Arbeit führen. Ein Scrum-Team besteht aus einer kleinen, funktionsübergreifenden, selbstverwalteten Einheit, die dafür verantwortlich ist, in jedem Sprint ein wertvolles Produktinkrement zu liefern.

Scrum gibt zwar eine Struktur vor, ist jedoch kein völlig starres System. So kann die konkrete Umsetzung des Frameworks an die individuellen Anforderungen eines Unternehmens angepasst werden. Es gibt viele Theorien darüber, welche konkreten Arbeitsweisen Scrum-Teams zum Erfolg führen.

Atlassian unterstützt agile Teams nun seit mehr als zehn Jahren bei der Bewältigung ihrer Aufgaben und wir haben die Erfahrung gemacht, dass eine klare Kommunikation, Transparenz sowie eine Hingabe für kontinuierliche Verbesserung stets im Mittelpunkt stehen sollten, gleich welches Framework genutzt wird.

Alles andere bleibt dem Team überlassen.

Welche Mitglieder hat ein Scrum-Team?

Ein Scrum-Team ist klein und flexibel und dafür zuständig, vorgesehene Produktinkremente bereitzustellen. Es ist normalerweise eine schlanke, eng verbundene Gruppe von ungefähr zehn Personen.

Aber sie ist oft groß genug, um eine beträchtliche Anzahl an Aufgaben innerhalb eines Sprints zu erledigen. In einem Scrum-Team sind drei Rollen erforderlich: Produktinhaber, Scrum Master und Entwicklerteam.

Der Scrum-Product Owner

Produktinhaber sind die Champions für ihr Produkt. Sie konzentrieren sich darauf, Unternehmens-, Kunden- und Marktanforderungen zu verstehen.

Diese Rolle gibt entsprechende Priorisierungen für die Aufgaben, die vom Entwicklerteam zu erledigen sind. Effektive Produktinhaber sind für Folgendes zuständig:

- Entwickeln und Verwalten des Produkt-Backlogs
- Nutzen von [Sprint-Backlog-Management](#), um das Produkt-Backlog zu [entwickeln](#), zu ordnen und transparent zu verwalten und dadurch eine effektive Planung und Priorisierung zu ermöglichen
- Zusammenarbeit mit dem Unternehmen und dem Team, um sicherzustellen, dass alle die Vorgänge im Produkt-Backlog verstehen
- Klare Anweisungen an das Team, welche Funktionen als Nächstes ausgeliefert werden sollen
- Entscheidung, wann das Produkt ausgeliefert werden soll, mit Tendenz zu einer häufigeren Auslieferung

Der Produktinhaber ist nicht immer der [Produktmanager](#). Produktinhaber stellen sicher, dass das Entwicklerteam dem Unternehmen den größtmöglichen Nutzen bringt.

Beim Produktinhaber muss es sich um eine Einzelperson handeln. Kein Entwicklerteam braucht gemischte Anweisungen von mehreren Produktinhabern.

Der Scrum Master

Scrum Master sind die Scrum-Champions in ihren Teams. Sie coachen Teams, Produktinhaber und das Unternehmen im Hinblick auf den Scrum-Prozess und suchen nach Möglichkeiten, ihr Handwerk zu verfeinern.

Eine wichtige Aufgabe des Scrum Masters ist es, die Effektivität des Scrum-Teams durch Coaching, Beseitigung von Hindernissen und Förderung von Scrum-Prozessen zu steigern. Dadurch können Teamleistung und Lieferung insgesamt verbessert werden.

Ein effektiver Scrum Master versteht die Arbeit des Teams genau und kann dem Team dabei helfen, seine Transparenz und seinen Lieferfluss zu optimieren.

Als leitender Moderator planen sie die erforderlichen Ressourcen (sowohl personelle als auch logistische). Diese Ressourcen decken alles ab, von der Sprintplanung bis hin zu [Stand-ups](#), Sprint-Reviews und [Sprint-Retrospektiven](#).

Das Scrum-Entwicklerteam

Scrum-Entwicklerteams sind echt produktiv. Was nachhaltige Entwicklungsverfahren angeht, sind sie wahre Champions.

Effektive Scrum-Teams arbeiten eng an einem gemeinsamen Standort zusammen und bestehen für gewöhnlich aus fünf bis sieben Mitgliedern. Die Größe des Teams lässt sich zum Beispiel mit der bekannten "Zwei-Pizza-Regel" von Amazon CEO Jeff Bezos ausloten: Ein Team sollte sich zwei Pizzas teilen können.

Die Teammitglieder sollten unterschiedliche Kompetenzen mitbringen und sich gegenseitig schulen, um durch Personen verursachte Engpässe bei der Auslieferung der Aufgaben zu vermeiden. Erfolgreiche Scrum-Teams sind in der Lage, sich selbst zu organisieren, und gehen ihre Projekte mit einer klaren "Wir"-Haltung an.

Alle Teammitglieder unterstützen sich gegenseitig, um einen erfolgreichen Sprint-Abschluss sicherzustellen. Das Scrum-Team legt den Plan für jeden Sprint fest.

Anhand ihres aus früheren Projekten bekannten Arbeitstempos prognostizieren sie, wie viel Arbeit sie während der Iteration erledigen können. Das Team nutzt seine bisherigen Leistungen, um seine Kapazitäten einzuschätzen und die Genauigkeit seiner Sprintplanung zu verbessern.

Da die Iterationen eine feste Dauer haben, erhält das Entwicklerteam wichtiges Feedback zu seinem Schätzungs- und Lieferprozess, wodurch seine Prognosen immer genauer werden. Auf Grundlage dieser Bewertung überlegt das Team sorgfältig, was im kommenden Sprint umgesetzt werden kann.

Was sind Scrum-Artefakte?

Scrum-Artefakte sind wichtige Informationen, die das Scrum-Team nutzt, um das Produkt zu definieren und festzulegen, welche Aufgaben zu seiner Erstellung erforderlich sind. Scrum umfasst drei Artefakte: ein Produkt-Backlog, ein Sprint-Backlog und ein Inkrement mit deiner [Definition von "erledigt"](#) (Definition of Done, DoD).

Das sind die drei Konstanten, auf die sich ein Scrum-Team im Laufe der Zeit und während [Sprints](#) beziehen sollte.

Produkt-Backlog

Dies ist die wichtigste Liste der Aufgaben, die der Produktinhaber oder Produktmanager erledigen und pflegen muss. Es ist eine dynamische Liste von Funktionen, Anforderungen, Verbesserungen und Fehlerbehebungen, die in das Sprint-Backlog einfließt.

Im Grunde genommen ist es die "To-do-Liste" des Teams.

Auf das Produkt-Backlog wird fortlaufend zurückgegriffen und es wird ständig neu priorisiert. Die Verwaltung übernimmt der Produktinhaber. Wenn Teams Erkenntnisse

gewinnen oder sich der Markt verändert, sind manche Aufgabenelemente womöglich nicht mehr relevant oder Probleme lassen sich auf andere Weise lösen.

Backlog-Aufgaben werden priorisiert, um den Kundennutzen zu maximieren und sicherzustellen, dass die bereitgestellten Funktionen die Kundenzufriedenheit und den langfristigen Geschäftswert direkt steigern.

Sprint-Backlog

Dies ist die Liste der Elemente, [User Stories](#) oder Fehlerbehebungen, die vom Entwicklerteam für die Implementierung im aktuellen Sprint-Zyklus ausgewählt wurden. Vor jedem Sprint wählt das Team während der Sprintplanung aus dem Backlog die Elemente aus, an denen es arbeiten wird.

Ein Sprint-Backlog kann flexibel sein und sich während eines Sprints weiterentwickeln. Das grundlegende Sprint-Ziel, das das Team mit dem aktuellen Sprint erreichen will, muss jedoch unverändert bleiben.

Inkrement (oder Sprintziel)

Inkrement, oder allgemein bekannt als Sprintziel, ist das nutzbare Endprodukt als Ergebnis eines Sprints. Bei Atlassian demonstrieren wir das *Inkrement* normalerweise während der Demo zum Ende des Sprints, bei der das Team präsentiert, was während des Sprints fertiggestellt wurde.

Dieser Begriff wird oft als die *Teamdefinition* von "Erledigt", als [Meilenstein](#), Sprintziel oder sogar als Vollversion oder als [ausgeliefertes Epic](#) bezeichnet. Es hängt nur davon ab, wie dein Team "Erledigt" definiert und wie du deine Sprintziele definierst.

Zum Beispiel releasen einige Teams am Ende jedes Sprints etwas für ihre Kunden. Ihre Definition von "erledigt" wäre demnach *geliefert*. Wie du siehst, kann dein Team viele Varianten selbst bestimmen, auch bei den Artefakten.

Daher ist es wichtig, offen für neue Wege zur Handhabung der Artefakte zu bleiben. Vielleicht belastet deine Definition von *erledigt* dein Team übermäßig und du musst sie noch einmal überdenken und eine neue Definition festlegen.

PROFITIPP

Gehe mit deinem Framework genauso agil um wie mit deinem Produkt. Nimm dir die nötige Zeit für Überprüfungen und notfalls auch für Anpassungen. Du solltest nichts erzwingen, nur um die Einheitlichkeit zu wahren.

Welche gängigen Zeremonien und Ereignisse gibt es bei Scrum?

Das Scrum-Framework umfasst Scrum-Praktiken, Zeremonien und Meetings, die Scrum-Teams regelmäßig durchführen. Wir beobachten große Unterschiede zwischen den [agilen Zeremonien](#) verschiedener Teams.

Wir empfehlen, zunächst zwei Sprints lang alle Zeremonien durchzugehen und erst danach zu entscheiden, was übernommen werden soll. Mithilfe einer kurzen Überprüfung kannst du dann feststellen, wo Anpassungen vorgenommen werden müssen. Wir stellen die wichtigsten Zeremonien vor, die Scrum-Teams nutzen können:

Organisieren des Backlogs

Dieses auch als [Backlog-Grooming](#) bezeichnete Ereignis unterliegt der Verantwortung des Produktinhabers. Die Hauptaufgabe des Produktinhabers besteht darin, das Produkt seiner Produktvision näher zu bringen und den Markt und die Kunden im Auge behalten.

Produktinhaber pflegen diese Liste anhand des Feedbacks von Benutzern und des Entwicklerteams, um Prioritäten zu setzen und die Liste relevant und jederzeit einsatzbereit zu halten.

Sprintplanung

Das gesamte Entwicklerteam plant während dieses Meetings die während des aktuellen Sprints auszuführenden Arbeiten (Umfang). Der Scrum Master leitet dieses Meeting, das als [Sprint-Planungsmeeting](#) bekannt ist.

Während dieser Besprechung legt das Team das [Sprintziel fest](#) und plant die auszuführenden Arbeiten. Spezifische User Storys werden dann aus dem Produkt-Backlog zum Sprint hinzugefügt.

Diese Storys sind immer in Einklang mit dem Ziel und müssen laut Einschätzung des Scrum-Teams realistisch sein (d. h., dass sie während des Sprints umgesetzt werden können). Am Ende des Meetings sollten alle Mitglieder des Scrum-Teams ein klares Verständnis davon haben, was im Sprint erreicht werden kann und wie das Inkrement realisierbar ist.

Sprint-Ausführung

Ein Sprint ist der Zeitraum, in dem das Scrum-Team zusammenarbeitet, um ein Inkrement abzuschließen. Diese zeitlich begrenzten Iterationen werden Sprints genannt und dauern in der Regel ein bis vier Wochen.

Die gängigste Sprint-Dauer beträgt zwei Wochen, doch es gibt auch Teams, die mit einer Woche oder einem Monat pro Inkrement besser zurechtkommen. Dave West von [Scrum.org](#) rät: Je komplexer die Aufgaben und je mehr unbekannte Faktoren vorhanden sind, desto kürzer sollte der Sprint sein.

Letztlich kann dein Team aber selbst über die Dauer entscheiden und bei Bedarf auch später noch Änderungen vornehmen, wenn sich die gewählte Dauer nicht bewährt. Während dieser Zeit kann der Umfang bei Bedarf zwischen dem Produktinhaber und dem Entwicklerteam neu ausgehandelt werden.

Das ist die Crux der empirischen Natur von Scrum. Alle Ereignisse, von der Planung bis zur Retrospektive, finden während des Sprints statt. Wenn ein bestimmtes Zeitintervall für einen Sprint festgelegt wurde, kann es während der gesamten Entwicklungsphase nicht mehr geändert werden.

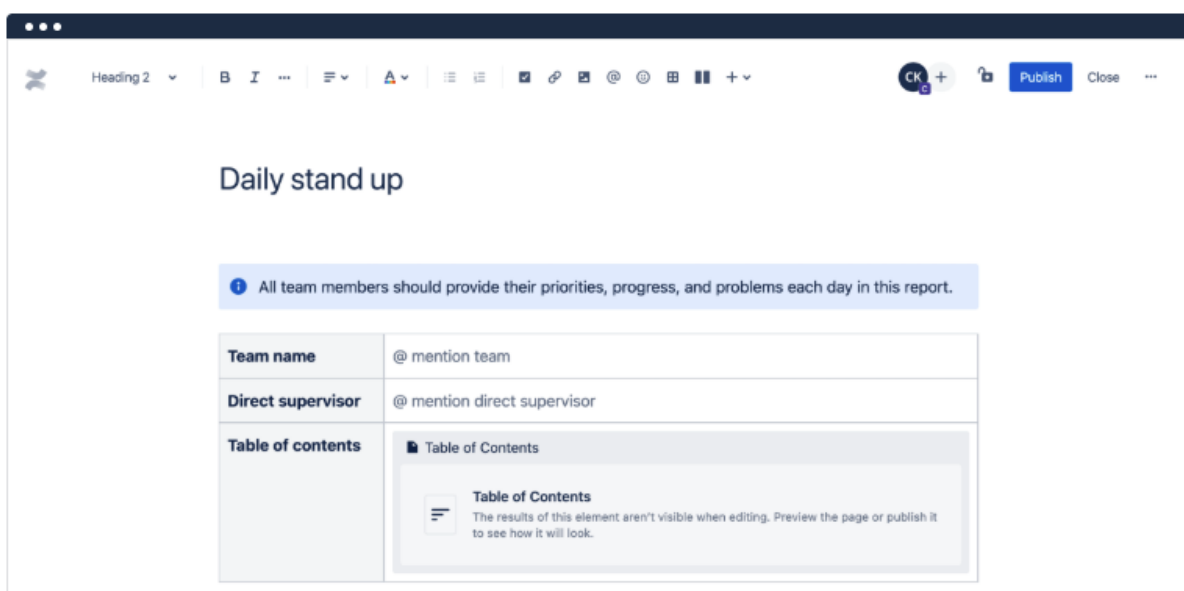
Dies hilft dem Team, aus vergangenen Erfahrungen zu lernen und diese Erkenntnisse auf künftige Sprints anzuwenden.

Daily Scrum-/Stand-up-Meetings

Zu diesem kurzen täglichen Meeting kommen alle zusammen (normalerweise gleich morgens). Die Daily Scrums haben eine feste Dauer und dienen dazu, die Fortschritte im Hinblick auf das Sprintziel zu überprüfen, die Aktivitäten im Team zu koordinieren und mögliche Hindernisse zu ermitteln.

Viele Teams beschränken das Meeting auf 15 Minuten. Das ist jedoch nur ein Orientierungswert. Man spricht auch von einem "Daily Stand-up", was verdeutlicht, dass die Besprechung wirklich nur sehr kurz sein sollte.

Im Daily Scrum sollen alle im Team auf denselben Stand gebracht werden, sich auf das Sprint-Ziel ausrichten und einen Plan für die nächsten 24 Stunden entwickeln. Du kannst eine [Vorlage fürs tägliche Stand-up-Meeting](#) verwenden, um die Meetings schneller vorzubereiten und zu organisieren.



The screenshot shows a web-based rich text editor interface. At the top, there's a toolbar with various editing tools. The main content area is titled "Daily stand up". Below the title, there's a blue informational box with a lightbulb icon and the text: "All team members should provide their priorities, progress, and problems each day in this report." Below this, there's a form with three sections: "Team name" with a placeholder "@ mention team", "Direct supervisor" with a placeholder "@ mention direct supervisor", and "Table of contents" which contains a "Table of Contents" heading and a note: "The results of this element aren't visible when editing. Preview the page or publish it to see how it will look."

Das Stand-up-Meeting bietet Gelegenheit, Bedenken zu äußern, die das Erreichen des Sprint-Ziels und mögliche Blocker betreffen. Eine gängige Methode für Stand-up-Meetings sieht vor, dass jedes Teammitglied drei Fragen zum Erreichen des Sprint-Ziels beantwortet:

- Woran habe ich gestern gearbeitet?
- Was habe ich für heute geplant?
- Gibt es irgendwelche Hindernisse?

Wir haben jedoch beobachtet, dass Teams dazu neigen, einfach ihre Kalendereinträge für den gestrigen und den kommenden Tag vorzulesen. Der Grundgedanke hinter Stand-up-Meetings ist, Ablenkungen durch Gespräche zu verringern, indem sie auf ein tägliches Meeting beschränkt werden, damit sich das Team den Rest des Tages auf seine eigentlichen Aufgaben konzentrieren kann.

Wenn das Meeting also zum reinen Kalendervorlesen wird, scheut euch nicht, für Schwung zu sorgen und kreativ zu werden.

Sprint-Review

Am Ende des Sprints kommt das Team in einer informellen Sitzung zusammen, um sich eine Demo des Inkrements anzusehen oder das Inkrement unter die Lupe zu nehmen. Das Entwicklerteam präsentiert die Backlog-Elemente, die jetzt *Erledigt* und für Feedback von Stakeholdern und Teamkollegen freigegeben sind.

Der Produktinhaber kann entscheiden, ob das Inkrement freigegeben wird, was in den meisten Fällen geschieht. In diesem Review-Meeting überarbeitet der Produktinhaber auch das Produkt-Backlog auf der Grundlage des aktuellen Sprints. Das Ergebnis kann dann in die nächste Sprintplanungssitzung einfließen.

Im Falle eines einmonatigen Sprints solltest du deinen [Sprint-Review](#) auf maximal vier Stunden beschränken.

Sprint-Retrospektive

Bei einer [Retrospektive](#) dokumentiert und bespricht das Team, was in einem Sprint, bei einem Projekt, bei bestimmten Personen oder Beziehungen, bei Tools oder auch bestimmten Zeremonien funktioniert hat und was nicht.

Das soll dem Team die Möglichkeit geben, sich nicht auf Misserfolge, sondern auf Erfolge zu konzentrieren und herauszustellen, was beim nächsten Mal besser laufen sollte.

Scrum-Werte

Im Jahr 2016 wurden fünf Scrum-Werte zum [Scrum-Leitfaden](#) hinzugefügt. Mithilfe dieser Werte werden die Arbeit, die Aktionen und das Verhalten von Scrum-Teams in eine bestimmte Richtung gelenkt. Effektive Zusammenarbeit, Engagement für die Aufgaben und die Einhaltung der Scrum-Prinzipien sind für Teams unerlässlich, um in einer agilen Umgebung erfolgreich zu sein.

Verpflichtung

Da Scrum-Teams klein und agil sind, ist jedes Teammitglied wichtig für den Erfolg des Teams. Alle Teammitglieder sollten sich Aufgaben widmen, die sie bewältigen können, anstatt sich zu viel zuzumuten.

Das Team sollte sich regelmäßig über den Arbeitsfortschritt austauschen, oftmals in Form von Stand-up-Meetings.

Mut

Für ein Scrum-Team bedeutet Mut, den Status Quo und alles, das dem Erfolg des Teams im Weg steht, zu hinterfragen. Die Mitglieder des Scrum-Teams sollten den Mut besitzen und sich sicher genug fühlen, neue Wege einzuschlagen.

Ein Scrum-Team sollte sich trauen und wohl dabei fühlen, offen über Hindernisse, den Projektfortschritt, Verzögerungen und dergleichen zu sprechen.

Schwerpunkt

Das Zentrum des Workflows für Scrum-Teams ist der Sprint – ein festgelegter Zeitraum, in dem das Team besonders fokussiert ist und ein bestimmtes Arbeitskontingent erledigt. Der Sprint schafft eine Struktur und ermöglicht gleichzeitig die Fokussierung darauf, die anstehenden Aufgaben zu erledigen.

Offenheit

Das tägliche Stand-up-Meeting fördert Offenheit, damit Teams sich ehrlich über laufende Arbeiten und Hindernisse austauschen können. Bei Atlassian lassen wir unsere Scrum-Teams oft folgende Fragen beantworten:

- Woran habe ich gestern gearbeitet?
- Woran arbeite ich heute?
- Welche Issues behindern mich?

Das hilft dabei, den Fortschritt hervorzuheben und Probleme zu ermitteln. Fortschritte zu teilen, stärkt auch das Team.

Respekt

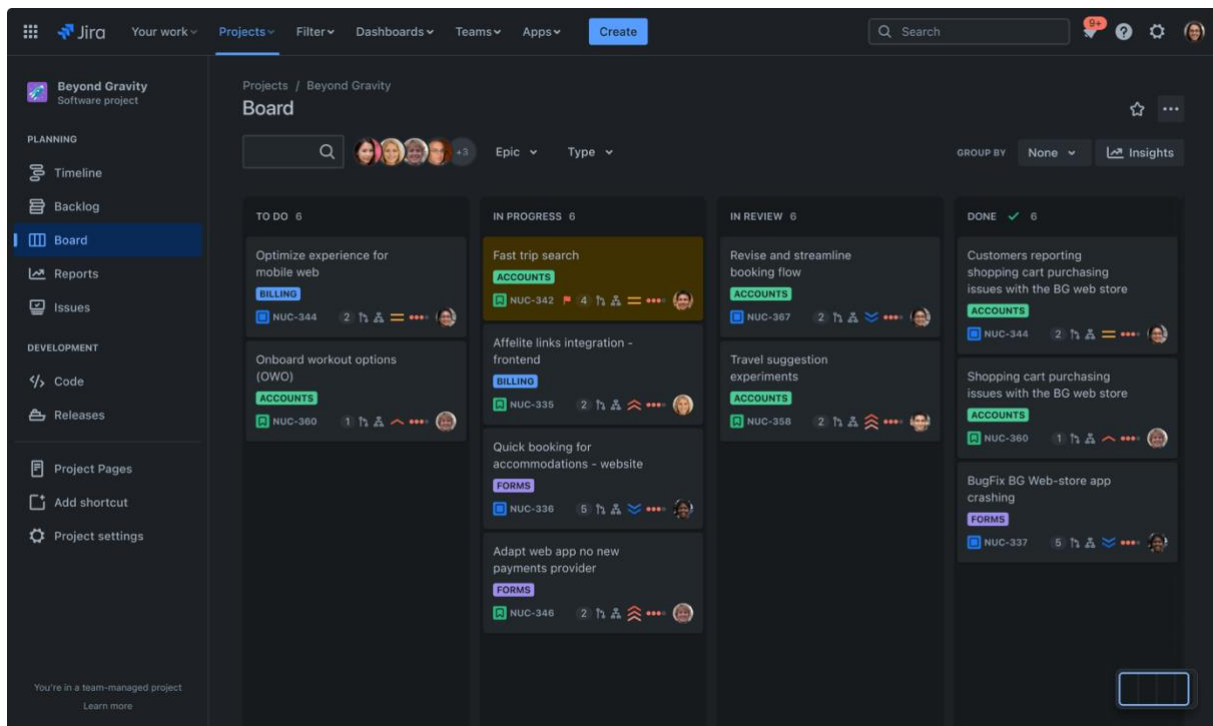
Die Stärke eines agilen Teams hängt von seiner Zusammenarbeit ab sowie von der Wertschätzung der Beiträge aller Teammitglieder zum Sprint. Im Team werden die Leistungen der Mitglieder gefeiert. Es herrscht ein respektvoller Umgang miteinander und mit dem Produktinhaber, den Stakeholdern sowie dem Scrum Master.

Scrum und Kanban

Scrum ist zwar das am weitesten verbreitete agile Framework, aber nicht das einzige verfügbare. Ein alternativer Ansatz ist [Kanban](#), bei dem Continuous Delivery und ein kontinuierlicher Arbeitsfluss anstelle von Iterationen mit fester Länge im Vordergrund stehen.

Merkmale von Kanban:

- Vorgänge werden auf einem [Board](#) visualisiert und mit WIP-Grenzen (Work-in-Progress-Grenzen) verwaltet.
- Teams nehmen nur dann neue Aufgaben an, wenn Kapazitäten frei werden, wodurch ein stetiger, kontinuierlicher Arbeitsfluss entsteht.
- Bei Kanban gibt es keine vorgegebenen Rollen oder Zeremonien, weswegen es gegenüber Scrum flexibler und "schlanker" ist.



Anwendungsfälle für Kanban:

- Ideal für Teams, bei denen kontinuierlich Aufgaben eingehen (beispielsweise Support oder Operations)
- Hilfreich, wenn sich Prioritäten häufig ändern und feste Sprint-Verpflichtungen schwer einzuhalten sind

Anwendungsfälle für Scrum:

- Am besten geeignet für die Produkt- oder Funktionsentwicklung, bei der eine schrittweise Bereitstellung, Struktur und Vorhersagbarkeit entscheidend sind
- Funktioniert gut bei Teams, die von klar definierten Rollen, Zeremonien und Sprint-Zielen profitieren

Gemischte Ansätze:

Manche Teams kombinieren Elemente aus beiden Frameworks zu etwas, das oft als **Scrumban** oder [Kanplan](#) (Kanban mit Backlog) bezeichnet wird. Ein solcher gemischter Ansatz ermöglicht es Teams, ein Backlog und die Planungsdisziplin von Scrum zu nutzen und gleichzeitig von der für Kanban typischen Flexibilität bei der Ausführung zu profitieren.

Scrum Alliance und Zertifizierungen

Die Scrum Alliance ist eine gute Anlaufstelle für alle, die ihr Verständnis der Scrum-Prinzipien vertiefen und ihre Laufbahn vorantreiben möchten. Als globale Organisation, die sich der Förderung des Scrum-Frameworks verschrieben hat, bietet die Scrum Alliance eine Reihe von Zertifizierungen und Schulungsprogrammen an.

Diese sind für Scrum Master, Produktinhaber und andere Scrum-Beteiligte konzipiert. Besonders beliebt sind die Zertifizierungen zum Certified Scrum Master (CSM) und Certified Scrum Product Owner (CSPO).

Sie vermitteln Fachleuten solide Grundkenntnisse der Scrum-Praktiken und -Werte. Mit diesen Zertifizierungen können Mitarbeiter zeigen, dass sie sich für agile Methoden einsetzen. Sie können auch Türen zu neuen Karrierechancen im Bereich Projekt- und Softwareentwicklung öffnen.

Die Scrum Alliance bietet Zugang zu einer aktiven Community, fortlaufenden Schulungen und branchenweit anerkannten Zertifizierungen und verhilft Einzelpersonen und Unternehmen so zum Scrum-Erfolg.

Zusammenarbeit mit einem Agile Coach

Die Einführung von Scrum und anderen agilen Methoden kann für jedes Unternehmen eine bedeutende Veränderung sein. Genau dabei kann ein Agile Coach helfen. Agile Coaches arbeiten eng mit den Scrum-Teams zusammen, um sie durch die Umstellung zu begleiten und die Teammitglieder dabei zu unterstützen, die Scrum-Prinzipien bei ihrer täglichen Arbeit anzuwenden.

Sie leiten Scrum-Ereignisse wie Sprintplanung und Retrospektiven und coachen Teams in Bezug auf Best Practices für die Zusammenarbeit und Kommunikation. Agile Coaches tragen auch entscheidend zur Beseitigung von Hindernissen bei, die den Fortschritt des Teams bremsen.

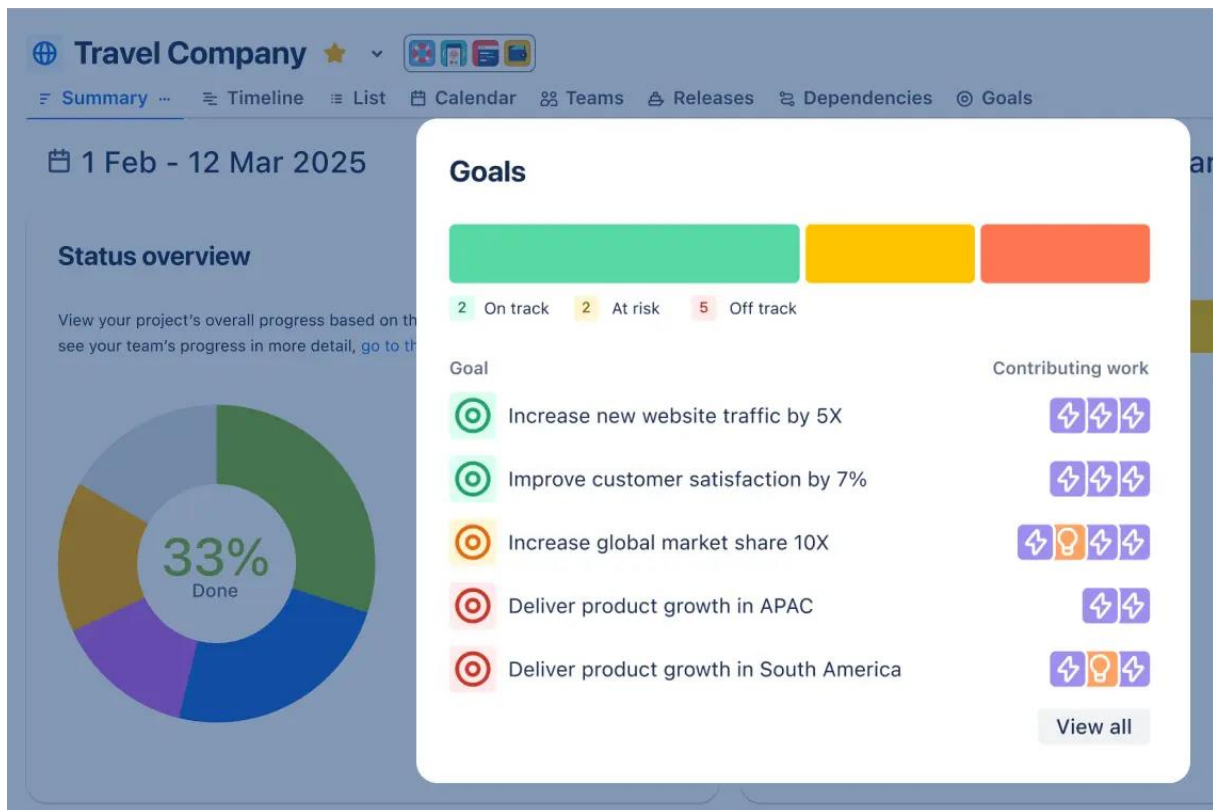
In erster Linie sollen Agile Coaches dafür sorgen, dass sich die Scrum-Teams auf wirklich wertschöpfende Aufgaben konzentrieren können. Mit maßgeschneiderten Schulungen und Unterstützung befähigen Agile Coaches die Teammitglieder, agile Praktiken anzuwenden.

So entsteht eine Kultur der kontinuierlichen Verbesserung und die Effektivität des Scrum-Teams steigt über mehrere Teams und Projekte hinweg.

Überwindung gängiger Herausforderungen bei Scrum

Selbst erfahrene Scrum-Teams stehen vor Herausforderungen wie Widerstand gegen Veränderungen, unklare Ziele und eingeschränktes Scrum-Training. Klein anzufangen, mit nur einem Projekt oder Team, kann helfen, diese Hürden zu überwinden.

Umfassende Schulungen und kontinuierliches Coaching stärken das Vertrauen in Scrum und agile Methoden. Klare Ziele, Rollen und Verantwortlichkeiten sorgen dafür, dass alle an einem Strang ziehen.



Regelmäßige Retrospektiven und Feedback-Sitzungen helfen Teams, zu reflektieren, Anpassungen vorzunehmen und Abläufe letztlich zu verbessern. Scrum-Teams gehen diese Herausforderungen frühzeitig an und bauen so Resilienz auf, arbeiten besser zusammen und haben mit dem Scrum-Framework Erfolg.

Messung des Erfolgs mit Scrum

Für kontinuierliche Verbesserung und für den Projekterfolg ist es unerlässlich, den Fortschritt und die Effektivität des Scrum-Teams zu verfolgen. Scrum-Teams ziehen zur Leistungsmessung oft die Sprint-Velocity, Burndown-Charts und die Kundenzufriedenheitswerte heran.

Die Überwachung von Vorlaufzeit, Durchlaufzeit und Fehlerdichte kann ebenfalls Einblick in den Workflow und die Produktqualität des Teams geben. Regelmäßige Sprint-Reviews und Retrospektiven sind wichtige Scrum-Ereignisse.

Bei diesen Reviews können die Teams ihre Leistung reflektieren, Bereiche mit Verbesserungsbedarf ermitteln und den erfolgreichen Sprint-Abschluss feiern. Wenn Scrum-Teams diese Indikatoren konsequent messen und auf Feedback reagieren, verbessern sie ihre Praktiken, schaffen einen höheren Kundennutzen und stellen den kontinuierlichen Erfolg mit dem Scrum-Framework sicher.

Erste Schritte mit Scrum

Das Scrum-Framework an sich ist einfach. Die Regeln, Artefakte, Ereignisse und Rollen sind leicht zu verstehen. Der Ansatz ist nur teilweise festgesetzt und trägt dazu bei, Unklarheiten im Entwicklungsprozess zu beseitigen.

Dabei bleibt den Unternehmen zugleich genügend Freiraum für eine individuelle Anpassung des Ansatzes. Scrum ist durch die Aufteilung komplexer Aufgaben in gut handhabbare User Storys ideal für schwierige Projekte geeignet.

Die klare Abgrenzung von Rollen und geplanten Ereignissen sorgt für Transparenz und fördert ein kollektives Verantwortungsgefühl im gesamten Entwicklungszyklus. Dank zügiger Releases bleibt das Team motiviert und die Zufriedenheit der Kunden ist sichergestellt, da Benutzer innerhalb kurzer Zeit Fortschritte erkennen können.

Jedoch muss das Entwicklerteam Zeit zur Einarbeitung einplanen, vor allem dann, wenn die Entwickler es gewohnt waren, mit einem typischen [Wasserfallmodell](#) zu arbeiten. Kleinere Iterationen, Daily Scrums, Sprint-Reviews und das Bestimmen eines Scrum Masters sind Konzepte, die neue Teams womöglich mit einem grundlegenden kulturellen Wandel konfrontieren.

Die langfristigen Vorteile wiegen jedoch den lernintensiven Einstieg auf. Die erfolgreiche Entwicklung komplexer Hardware- und Softwareprodukte in verschiedenen Branchen und Sektoren macht Scrum [auch für dein Unternehmen](#) zum attraktiven Framework.

Weitere Informationen zu Scrum in Jira erfährst du in [diesem Tutorial](#).

Beispiel Folgen :

- Viele Meetings zwischen Teams nötig
- Integration dauert lange
- Product Owner überlastet mit Priorisierung
- Management sieht nur Fortschritt einzelner Teams, nicht Gesamtplattform

Unabhängige, aber koordinierte Teams

- Team A arbeitet z. B. an Zahlungsmodul
- Team B an Warenkorb-UI
- Team C an Suchfunktion
- Alle Tasks sind so geplant, dass sie **einander minimal blockieren**

Projektbeispiel: Online-Shop

Wir haben drei Teams: **Team A (Zahlungsmodul)**, **Team B (Warenkorb-UI)**, **Team C (Suchfunktion)**. Jedes Team hat ein eigenes Feature, aber sie sind teilweise voneinander abhängig.

Team A – Zahlungsmodul

Feature:

- Implementierung der gesamten Zahlungsabwicklung im System.

Aufgaben im Detail:

1. Datenbank entwerfen und implementieren

- Erstellen von Tabellen für Transaktionen, Kundenkonten, Bestellungen, Rückerstattungen usw.
- Definieren von Beziehungen zwischen den Tabellen (z. B. Kunde → Bestellung → Zahlung).
- Sicherstellen, dass die Datenbank **konsistent, erweiterbar und performant** ist.
- Dokumentation der Datenstrukturen, damit andere Teams sie korrekt nutzen können.

2. Zahlungslogik entwickeln

- Autorisierung von Zahlungen (z. B. Kreditkarte, PayPal).
- Verbuchung der Zahlungen in der Datenbank.
- Rückerstattungen und Fehlerbehandlung.
- Logging aller Zahlungsaktionen für Nachvollziehbarkeit.
- Sicherheitsmaßnahmen, z. B. Verschlüsselung von sensiblen Daten.

3. APIs / Schnittstellen bereitstellen

- Bereitstellung von API-Endpunkten für andere Teams: z. B. Team B (Warenkorb) oder Team C (Suchfunktion).
- Dokumentation der Schnittstellen, damit andere Teams genau wissen, wie sie Zahlungen auslösen oder Daten abrufen können.
- Bereitstellung von **Stubs oder Mock-APIs**, damit andere Teams schon arbeiten können, bevor Team A alles final implementiert hat.

Problem / Herausforderung:

- Team A muss **die Datenbankstrukturen und Schnittstellen mit Team B abstimmen**, sonst kann Team B keine korrekten Transaktionen im Warenkorb abbilden.
 - Es ist also nicht nur reine Datenbankarbeit – Team A ist **der zentrale Baustein**, auf den andere Teams zugreifen.
-

Team B – Warenkorb-UI

Feature:

- Benutzeroberfläche für den Warenkorb, wo Kunden Produkte auswählen, ändern oder bestellen können.

Aufgaben im Detail:

1. **Warenkorb-Funktionalität entwickeln**
 - Hinzufügen/Entfernen von Produkten.
 - Berechnung von Gesamtsumme, Steuern, Versandkosten.
 - Anzeige von aktuellen Beständen (falls Produkt-API integriert).
2. **Zahlungsintegration**
 - Team B muss Zahlungen auslösen, sobald der Kunde auf „Kaufen“ klickt.
 - Dafür braucht Team B die **Datenbankstrukturen und APIs von Team A**, z. B. Transaktionen oder Stubs.

Problem / Herausforderung:

- **Abhängigkeit zu Team A:** Team B kann den Warenkorb nicht vollständig testen, solange Team A die Datenbank oder Zahlungs-APIs nicht bereitstellt.
 - Lösung: Team B arbeitet zunächst mit **Mock-Daten oder Stubs**, um die Entwicklung fortzusetzen.
-

Team C – Suchfunktion

Feature:

- Implementierung der Produktsuche, Filter und Anzeige der Ergebnisse.

Aufgaben im Detail:

1. **Suchlogik entwickeln**
 - Volltextsuche, Filter nach Kategorie, Preis, Verfügbarkeit.
 - Integration von Datenstrukturen anderer Teams, z. B. Produktinformationen oder Zahlungsstatus.
2. **API-Integration**
 - Zugriff auf APIs von Team A (Zahlungen) und Team B (Warenkorb).
 - Anzeigen von Produkten, die aktuell verfügbar sind, und ggf. Status von Transaktionen.

Problem / Herausforderung:

- **Abhängigkeit zu Team A/B:** Team C benötigt funktionierende APIs, um echte Daten anzuzeigen.
- Lösung: Team C kann zunächst **mit Stubs oder Mock-Daten** arbeiten und später integrieren.

LeSS (Large Scale Scrum) ist ein [agiles Framework](#) für die [Organisation mehrerer Scrum Teams](#). Dabei ist LeSS auf zwei bis acht Teams ausgelegt, wohingegen LeSS Huge einen Ansatz für mehr als acht Teams bietet. Das ultimative Ziel einer LeSS Implementierung ist, Komplexität zu reduzieren und die Agilität einer Organisation zu maximieren, damit immer die werthaltigste Arbeit erledigt werden kann.

In diesem Artikel erkläre ich dir den Aufbau des LeSS Frameworks und wie es gelingt, Scrum über mehrere Teams und auch hunderte von Mitarbeitern zu skalieren.

Alternative skalierte Scrum Frameworks zum LeSS Ansatz bieten das [Scrum@Scale Modell](#) von Jeff Sutherland sowie [Nexus](#) von Ken Schaber.

Inhaltsverzeichnis

Was ist LeSS?

LeSS ist ein Vorgehensmodell zur Skalierung von Scrum. Oder um es mit einem der zentralen Prinzipien von LeSS zu formulieren: "LeSS ist Scrum". Dieses Eingeständnis ist für das Verständnis des LeSS Frameworks elementar. Denn damit bleiben zentrale Punkte aus dem Scrum Framework trotz oder gerade wegen der Skalierung erhalten:

- Es gibt nur **einen Product Owner** und **einen Product Backlog**
- Es gibt eine, **für alle Teams gültige, Definition of Done**
- Teams sind **crossfunktional** besetzt und arbeiten "**self managed**"
- Alle Teams arbeiten synchron im gleichen **Sprint-Rhythmus**
- Teams arbeiten an einem **gemeinsamen Inkrement**

Basierend auf diesen aus Scrum übernommenen Ansätzen definiert LeSS zusätzliche Events und erweitert den Kompetenzbereich existierender Scrum-Rollen. Das heißt, LeSS ist eine Erweiterung der Rollen, Meetings und Artefakte des Scrum Frameworks.

Fragen :

Frage 1

Welche der folgenden Rollen gehört **nicht** zu Scrum?

- A) Product Owner
- B) Scrum Master
- C) Entwicklerteam
- D) Projektmanager

Richtige Antwort: D

Frage 2

Was ist das Hauptziel des **Sprint-Backlogs**?

- A) Den Fortschritt aller Teams auf Unternehmensebene zu zeigen
- B) Die Arbeit zu planen, die im kommenden Sprint erledigt werden soll

- C) Stakeholder über den Gesamtfortschritt informieren
- D) Alle Bugs aus dem Product Backlog löschen

Richtige Antwort: B

Frage 3

Welche Aussage über Large Scale Scrum (LeSS) ist korrekt?

- A) LeSS eignet sich nur für ein einzelnes Team.
- B) LeSS ist ein Vorgehensmodell zur Skalierung von Scrum.
- C) LeSS ersetzt Scrum komplett durch ein neues Framework.
- D) LeSS arbeitet ohne Product Owner.

Richtige Antwort: B

Frage 4

Für wie viele Teams ist LeSS typischerweise ausgelegt?

- A) 1 Team
- B) 2–8 Teams
- C) 10–20 Teams
- D) 50 Teams

Richtige Antwort: B

Frage 5

Welches Ziel verfolgt LeSS?

- A) Die Anzahl der Meetings zu erhöhen
- B) Komplexität zu reduzieren und die Agilität einer Organisation zu maximieren
- C) Nur einzelne Teams zu bewerten
- D) Den Product Owner überflüssig zu machen

Richtige Antwort: B

Frage6

Welche der folgenden Aussagen über Rollen und Artefakte in LeSS ist korrekt?

- A) Jedes Team hat einen eigenen Product Owner
- B) Es gibt einen Product Owner für alle Teams
- C) Jedes Team hat einen eigenen Product Backlog
- D) Teams arbeiten ohne Definition of Done

Richtige Antwort: B